

Отчет к лабораторной работе №4. ИГИ.  
Работа с файлами, классами, сериализаторами, регулярными выражениями и  
стандартными библиотеками.

Студента гр. 453504  
Романова Николая

**Вариант 23**

**Задание 1.** Исходные данные представляют собой словарь. Необходимо поместить их в файл, используя сериализатор. Организовать считывание данных, поиск, сортировку в соответствии с индивидуальным заданием. Обязательно использовать классы. Реализуйте два варианта: 1)формат файлов CSV; 2)модуль pickle.

Индивидуальное задание: Реализуйте оценки за год. Посчитайте процент и качество успеваемости в классе за год, составьте списки неуспевающих и отличников. Выведите информацию о школьнике, введенном с клавиатуры

Код задания 1:

```
"""
Brief purpose: Module for Task 1. Works with dictionaries, CSV, and
Pickle serialization.

Lab number and title: Lab 4 - Working with files, classes, serializers,
regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 15.03.2026
"""

import csv

import pickle

from validation import Validation
```

```

class Students:

    """Class representing a collection of students and their grades."""

    __default = {

        "Nikolay": [8, 8, 10],

        "Vladislav": [4, 10, 8],

        "Sergey": [9, 2, 2],

        "Maksim": [8, 10, 9],

        "Ivan": [5, 6, 4]

    }

    @classmethod

    def default(cls):

        """Class method to return default students data."""

        return cls.__default

    def __init__(self):

        """Magic method: constructor to initialize dynamic
attributes."""

        self.__students = Students.default()

    @property

    def students(self):

        """Getter for students dictionary (Requirement 4)."""

        return self.__students

```

```

@students.setter

def students(self, dict_obj):

    """Setter for students dictionary (Requirement 4)."""

    self.__students = dict_obj

@property

def grades_type(self):

    """Getter returning the number of subjects per student."""

    return len(next(iter(self.__students.values())))

class TaskStorage:

    """Static class for storing configuration and runtime data."""

    filename_csv = "task1.csv"

    filename_txt = "task1.txt"

    study = Students()

    excellent_grade = 8

    satisfactory_grade = 4

    grades_count = study.grades_type * len(study.students)

    satisfactory_grades_count = 0

    excellent_grades_count = 0

    failing_students = []

    excellent_students =[]

```

```

    @classmethod

    def clear(cls):

        """Class method to clear lists and counters before new
operations."""

        cls.satisfactory_grades_count = 0

        cls.excellent_grades_count = 0

        cls.failing_students = []

        cls.excellent_students = []

class CSVService:

    """Service class for CSV operations."""

    @staticmethod

    def serialization():

        """Serializes dictionary data into a CSV file."""

        TaskStorage.clear()

        with open(TaskStorage.filename_csv, "w", encoding="utf-8",
newline="") as fh:

            writer = csv.DictWriter(fh, fieldnames=["name", "rus",
"phys", "math"], quoting=csv.QUOTE_ALL)

            writer.writeheader()

            for name, grades in
sorted(TaskStorage.study.students.items()):

                writer.writerow(dict(name=name, rus=grades[0],
phys=grades[1], math=grades[2]))

```

```

@staticmethod

def deserialization():

    """Deserializes CSV file and calculates statistics."""

    with open(TaskStorage.filename_csv, 'r', encoding="utf-8",
newline="") as fh:

        reader = list(csv.DictReader(fh))

        for student in reader:

            student_grades = [int(student['rus']),
int(student['phys']), int(student['math'])]

            is_excellent = True

            is_failing = False

            for grade in student_grades:

                if grade >= TaskStorage.excellent_grade:

                    TaskStorage.excellent_grades_count += 1

                    TaskStorage.satisfactory_grades_count += 1

                elif grade >= TaskStorage.satisfactory_grade:

                    is_excellent = False

                    TaskStorage.satisfactory_grades_count += 1

                else:

                    is_excellent = False

                    is_failing = True

            if is_excellent:

```

```

        TaskStorage.excellent_students.append(student)

    elif is_failing:

        TaskStorage.failing_students.append(student)

class PickleService:

    """Service class for Pickle operations."""

    @staticmethod

    def serialization():

        """Serializes dictionary data into a binary file using
Pickle."""

        TaskStorage.clear()

        with open(TaskStorage.filename_txt, "wb") as fh:

            pickle.dump(TaskStorage.study.students, fh)

    @staticmethod

    def deserialization():

        """Deserializes Pickle file and calculates statistics."""

        with open(TaskStorage.filename_txt, 'rb') as fh:

            reader = pickle.load(fh)

            for student_name, grades in reader.items():

                is_excellent = True

                is_failing = False

```

```

        for grade in grades:

            if grade >= TaskStorage.excellent_grade:

                TaskStorage.excellent_grades_count += 1

                TaskStorage.satisfactory_grades_count += 1

            elif grade >= TaskStorage.satisfactory_grade:

                is_excellent = False

                TaskStorage.satisfactory_grades_count += 1

            else:

                is_excellent = False

                is_failing = True

        if is_excellent:

            TaskStorage.excellent_students.append({student_name:
grades})

        elif is_failing:

            TaskStorage.failing_students.append({student_name:
grades})

class ConsoleService:

    """Service for displaying data in a user-friendly format."""

    @staticmethod

    def printStatisticsPerformance():

        """Prints the percentage of satisfactory performance."""

```

```

        perc = TaskStorage.satisfactory_grades_count * 100 /
TaskStorage.grades_count

        print(f"Overall performance (satisfactory+): {perc:.1f}%")

    @staticmethod
    def printQualityPerformance():

        """Prints the percentage of excellent performance."""

        perc = TaskStorage.excellent_grades_count * 100 /
TaskStorage.grades_count

        print(f"Quality performance (excellent): {perc:.1f}%")

    @staticmethod
    def printExcellentStudents():

        """Prints the list of excellent students."""

        print(f"Excellent students: {TaskStorage.excellent_students}")

    @staticmethod
    def printFailingStudents():

        """Prints the list of failing students."""

        print(f"Failing students: {TaskStorage.failing_students}")

class SearchMixin:

    """Mixin class to add formatted print capabilities (Requirement 4:
Mixin)."""

    def print_result(self, result):

```

```

    if result:

        print(f"[Search Success] Found: {result}")

    else:

        print("[Search Failed] Student not found.")


class BaseSearch(SearchMixin):

    """Base class for searching students. Demonstrates Inheritance."""

    def __init__(self, filename, name):

        """Initializes protected dynamic attributes."""

        self._filename = filename

        self._name = name

    def findStudent(self):

        """Abstract method to be overridden (Polymorphism)."""

        raise NotImplementedError("This method must be overridden in
subclasses.")

    def __str__(self):

        """Magic method (Requirement 4) to return string
representation."""

        return str(self.findStudent())


class CSVSearch(BaseSearch):

```

```

"""Class to search within CSV files. Inherits from BaseSearch."""

def __init__(self, filename, name):
    super().__init__(filename, name)

def findStudent(self):
    """Polymorphism: overriding the base method for CSV specific
    logic."""
    with open(self._filename, "r", encoding="utf-8") as fh:
        reader = csv.DictReader(fh)
        for student in reader:
            if student["name"] == self._name:
                return student
        return None

class PickleSearch(BaseSearch):
    """Class to search within Pickle files. Inherits from BaseSearch."""

    def __init__(self, filename, name):
        super().__init__(filename, name)

    def findStudent(self):
        """Polymorphism: overriding the base method for Pickle specific
        logic."""
        with open(self._filename, "rb") as fh:

```

```

        reader = pickle.load(fh)

        if not reader.get(self._name):

            return None

        return {self._name: reader.get(self._name)}

class SearchService:

    """Service to prompt user and initiate search."""

    @staticmethod

    def searchCSV():

        """Prompts user and searches via CSV."""

        name = Validation.inputStr("Enter student name to search in CSV:")

        search_obj = CSVSearch(TaskStorage.filename_csv, name)

        search_obj.print_result(search_obj.findStudent()) # Using Mixin
method

    @staticmethod

    def searchPickle():

        """Prompts user and searches via Pickle."""

        name = Validation.inputStr("Enter student name to search in
Pickle: ")

        search_obj = PickleSearch(TaskStorage.filename_txt, name)

        search_obj.print_result(search_obj.findStudent()) # Using Mixin
method

```

```

class Service:

    """Main presentation layer coordinating Task 1 logic."""

    @staticmethod

    def presentation():

        """Runs the demonstration of CSV and Pickle processing."""

        print("\n--- [ CSV PROCESSING ] ---")

        CSVService.serialization()

        CSVService.deserialization()

        ConsoleService.printStatisticsPerformance()

        ConsoleService.printQualityPerformance()

        ConsoleService.printExcellentStudents()

        ConsoleService.printFailingStudents()

        SearchService.searchCSV()

        print("\n--- [ PICKLE PROCESSING ] ---")

        PickleService.serialization()

        PickleService.deserialization()

        ConsoleService.printStatisticsPerformance()

        ConsoleService.printQualityPerformance()

```

```
ConsoleService.printExcellentStudents()

ConsoleService.printFailingStudents()

SearchService.searchPickle()
```

### Демонстрация задания 1:

```
--- [ CSV PROCESSING ] ---
Overall performance (satisfactory+): 86.7%
Quality performance (excellent): 60.0%
Excellent students: [{'name': 'Maksim', 'rus': '8', 'phys': '10', 'math': '9'}, {'name': 'Nikolay', 'rus': '8', 'phys': '8', 'math': '10'}]
Failing students: [{'name': 'Sergey', 'rus': '9', 'phys': '2', 'math': '2'}]
Enter student name to search in CSV: Nikolay
[Search Success] Found: {'name': 'Nikolay', 'rus': '8', 'phys': '8', 'math': '10'}

--- [ PICKLE PROCESSING ] ---
Overall performance (satisfactory+): 86.7%
Quality performance (excellent): 60.0%
Excellent students: [{'Nikolay': [8, 8, 10]}, {'Maksim': [8, 10, 9]}]
Failing students: [{'Sergey': [9, 2, 2]}]
Enter student name to search in Pickle: Sergey
[Search Success] Found: {'Sergey': [9, 2, 2]}
```

```
task1.csv > data
1  "name", "rus", "phys", "math"
2  "Ivan", "5", "6", "4"
3  "Maksim", "8", "10", "9"
4  "Nikolay", "8", "8", "10"
5  "Sergey", "9", "2", "2"
6  "Vladislav", "4", "10", "8"
7  |
```

task1.csv

task1.py

task1.txt

**Задание 2.** В соответствии с заданием своего варианта составить программу для анализа текста. Считать из исходного файла текст. Используя регулярные выражения получить искомую информацию (см. условие), вывести ее на экран и сохранить в другой файл. Заархивировать файл с результатом с помощью модуля zipfile и обеспечить получение информации о файле в архиве.

Также выполнить общее задание

- определить и сохранить в файл с результатами:
- количество предложений в тексте;
- количество предложений в тексте каждого вида отдельно (повествовательные, вопросительные и побудительные);
- среднюю длину предложения в символах (считаются только слова);
- среднее количество смайликов в заданном тексте.

Смайликом будем считать последовательность символов, удовлетворяющую условиям: первым символом является либо «;» (точка с запятой) либо «:» (двоеточие) ровно один раз; далее может идти символ «-» (минус) сколько угодно раз (в том числе символ минус может идти ноль раз); в конце обязательно идет некоторое количество (не меньше одной) одинаковых скобок из следующего набора: «(», «)», «[», «]»; внутри смайлика не может встречаться никаких других символов. Например, эта последовательность является смайликом: «;-----[[[[[[[». Эти последовательности смайликами не являются: «]», «;-», «:», «)».

Индивидуальное задание: Вывести исходный текст, заменив пробелы символом, введенным с клавиатуры. Определить, является ли данная строка GUID со скобками или без скобок. GUID – это строка, состоящая из 8, 4, 4, 4, 12 шестнадцатеричных цифр, разделенных тире. Пример правильного выражения: e02fd0e4-00fd-090A-ca30-0d00a0038ba0. Пример неправильного выражения: e02fd0e400fd090Aca300d00a0038ba0. определить число слов, длина которых равна 3 символа; найти слова, у которых количество гласных равно количеству согласных и их порядковые номера; вывести слова в порядке убывания их длин.

Код задания 2:

```
"""
Brief purpose: Module for Task 2. Text analysis using Regular
Expressions, File I/O, and Zip archives.
Lab number and title: Lab 4 - Working with files, classes, serializers,
regular expressions, and standard libraries
Version: 1.0
Developer: Romanov Nikolay Viktorovich
Date: 16.03.2026
"""
```

```

import os
import re
from zipfile import ZipFile
from validation import Validation

class TaskStorage:
    """Static class for storing file paths and configuration data."""
    filename = "task2.txt"
    filename_result = "task2_result.txt"
    zipname = "task2.zip"
    data = ""

class BaseFileHandler:
    """Base class demonstrating OOP inheritance for file management."""

    def __init__(self, filename):
        """Initializes the base file handler using a protected
attribute."""
        self._filename = filename

    def get_info(self):
        """Abstract method to demonstrate polymorphism in subclasses."""
        raise NotImplementedError("Method get_info() must be
overridden.")

class FileManager(BaseFileHandler):
    """Class for reading and writing text files. Inherits from
BaseFileHandler."""

    def __init__(self, filename):
        super().__init__(filename)

    def read_file(self):
        """Reads text from the file."""
        if not os.path.exists(self._filename):
            print(f"Error: File '{self._filename}' not found.")
            return ""
        with open(self._filename, "r", encoding="utf-8") as fh:
            return fh.read()

    def write_results(self, results_dict):
        """Writes analysis results to the file."""

```

```

        with open(self._filename, "w", encoding="utf-8") as fh:
            for key, value in results_dict.items():
                fh.write(f"{key}: {value}\n")

    def get_info(self):
        """Polymorphism: Overrides base method to return text file
size."""
        size = os.path.getsize(self._filename) if
os.path.exists(self._filename) else 0
        return f"[FileManager Info] File '{self._filename}' is {size}
bytes."

class ZipManager(BaseFileHandler):
    """Class for creating and managing zip archives. Inherits from
BaseFileHandler."""

    def __init__(self, zipname):
        super().__init__(zipname)

    def create_zip(self, file_to_zip):
        """Archives the specified file into a ZIP archive."""
        if os.path.exists(file_to_zip):
            with ZipFile(self._filename, "w") as zipf:
                zipf.write(file_to_zip)
                print(f"File '{file_to_zip}' successfully archived to
'{self._filename}'.")

        # Detailed info about the first file in archive
    def get_info(self):
        """Polymorphism: Overrides base method to return archive
contents."""
        if not os.path.exists(self._filename):
            return f"[ZipManager Info] Archive '{self._filename}' does
not exist."

        with ZipFile(self._filename, "r") as zipf:
            files_in_zip = zipf.namelist()
            if files_in_zip:
                info = zipf.getinfo(files_in_zip[0])
                return (f"[ZipManager Info] Archive contains:
{files_in_zip}. "

```

```

        f"\nFile '{info.filename}' compressed size:
{info.compress_size} bytes.")
        return "[ZipManager Info] Archive is empty."

class IndividualTaskService:
    """Service for handling individual variant tasks using regex."""

    @staticmethod
    def replace_space(text):
        char_replace = Validation.inputStr("Enter symbol to replace
spaces in text: ")
        return re.sub(r"\s", char_replace, text)

    @staticmethod
    def check_guid(text):
        """Checks if the text contains a valid GUID."""
        pattern =
r'[\(\{\[\]?[a-fA-F0-9]{8}-[a-fA-F0-9]{4}-[a-fA-F0-9]{4}-[a-fA-F0-9]{4}-
[a-fA-F0-9]{12}[\)\}\]\]}?'
        return bool(re.search(pattern, text))

    @staticmethod
    def get_count_3_len(text):
        """Returns the number of 3-letter words."""
        return len(re.findall(r"\b[a-zA-Za-яA-ЯёЁ]{3}\b", text))

    @staticmethod
    def find_words_with_equal_vowels_consonants(text):
        """Finds words where the number of vowels equals consonants."""
        vowels = r'aeёiuoyэюяaeiouy'
        consonants = r'бвгджзйклмнпрстфхцчшщbcd fghjklmnpqrstvwxyz'

        words = re.findall(r'\b[a-яA-Яa-zA-ZёЁ]+\b', text)
        result = []

        for idx, word in enumerate(words, 1):
            word_lower = word.lower()
            vowel_count = len(re.findall(f'[{vowels}]', word_lower))
            consonant_count = len(re.findall(f'[{consonants}]',
word_lower))

            if vowel_count == consonant_count and vowel_count > 0:

```

```

        result.append(f"{idx}: {word}")

    return result

@staticmethod
def sort_words(text):
    """Sorts all words in the text by length in descending order."""
    words = re.findall(r"\b[a-zA-Za-яA-ЯёЁ]+\b", text)
    return sorted(words, key=len, reverse=True)

class CommonTaskService:
    """Service for handling common analysis tasks using regex."""

    @staticmethod
    def get_count_sentence(text):
        """Counts the total number of sentences."""
        return len(re.findall(r'([^.!?...]+)[.!?...]+', text))

    @staticmethod
    def get_exclamatory_count(text):
        """Counts exclamatory sentences (!)."""
        return len(re.findall(r'([^.!?...]+)[!]+', text))

    @staticmethod
    def get_declarative_count(text):
        """Counts declarative sentences (.)."""
        return len(re.findall(r'([^.!?...]+)[.]+', text))

    @staticmethod
    def get_interrogative_count(text):
        """Counts interrogative sentences (?)."""
        return len(re.findall(r'([^.!?...]+)[?]+', text))

    @staticmethod
    def get_average_len_sentence(text):
        """Calculates average sentence length in words."""
        sentences = re.findall(r'([^.!?...]+)[.!?...]+', text)
        if not sentences: return 0

        total_words = sum(len(re.findall(r"\b[a-zA-Za-яA-ЯёЁ]+\b", s))
        for s in sentences)
        return total_words / len(sentences)

```

```

    @staticmethod
    def get_average_len_word(text):
        """Calculates average word length in characters."""
        words = re.findall(r"\b[a-zA-Za-яА-ЯёЁ]+\b", text)
        if not words: return 0

        total_chars = sum(len(word) for word in words)
        return total_chars / len(words)

    @staticmethod
    def get_count_emoji(text):
        """Counts specific emojis like :-), ;-[[, :] """
        return len(re.findall(r'[;:] [-]* (?: \ ( + | \ ) + | \ [+ | \ ] + ) ', text))

class Service:
    """Main presentation layer coordinating Task 2 logic."""

    @staticmethod
    def presentation():
        """Executes text analysis, prints results, and saves them to
        archive."""
        print("\n--- [ TEXT ANALYSIS (TASK 2) ] ---")

        reader = FileManager(TaskStorage.filename)
        TaskStorage.data = reader.read_file()

        if not TaskStorage.data:
            return

        # Replace text based on user input
        replaced_text =
IndividualTaskService.replace_space(TaskStorage.data)
        print("\n[Preview of Text with Replaced Spaces]:")
        print(replaced_text[:100] + "... \n")

        # Gathering all statistics
        results = {
            "Contains GUID":
IndividualTaskService.check_guid(TaskStorage.data),
            "Count of 3-letter words":
IndividualTaskService.get_count_3_len(TaskStorage.data),

```

```

        "Words with equal vowels/consonants":
IndividualTaskService.find_words_with_equal_vowels_consonants(TaskStorage.data),
        "Total sentences":
CommonTaskService.get_count_sentence(TaskStorage.data),
        "Declarative sentences":
CommonTaskService.get_declarative_count(TaskStorage.data),
        "Interrogative sentences":
CommonTaskService.get_interrogative_count(TaskStorage.data),
        "Exclamatory sentences":
CommonTaskService.get_exclamatory_count(TaskStorage.data),
        "Average sentence length (words)":
round(CommonTaskService.get_average_len_sentence(TaskStorage.data), 2),
        "Average word length (chars)":
round(CommonTaskService.get_average_len_word(TaskStorage.data), 2),
        "Count of emojis":
CommonTaskService.get_count_emoji(TaskStorage.data),
        "Sorted words by length (preview)":
IndividualTaskService.sort_words(TaskStorage.data)
    }

    # Print results
    for key, value in results.items():
        print(f"{key:.<40}: {value}")

    # Save results to text file
    writer = FileManager(TaskStorage.filename_result)
    writer.write_results(results)
    print("\n" + writer.get_info())

    # Archive the result file
    zipper = ZipManager(TaskStorage.zipname)
    zipper.create_zip(TaskStorage.filename_result)
    print(zipper.get_info())

```

Демонстрация задания 2:

```

--- [ TEXT ANALYSIS (TASK 2) ] ---
Enter symbol to replace spaces in text: =

[Preview of Text with Replaced Spaces]:
Hello=there!=How=are=you=doing=today?=This=is=a=simple=test=file.=I=love=programming=in=Python=:-)=I...

Contains GUID.....: True
Count of 3-letter words.....: 18
Words with equal vowels/consonants.....: ['9: is', '13: file', '15: love', '17: in', '19: I
t', '20: is', '21: very', '28: we', '31: Here', '32: is', '35: identifier', '39: We', '40: n
eed', '41: some', '45: like', '51: like', '52: mama', '53: papa', '55: case', '56: have', '6
6: am', '69: go']
Total sentences.....: 11
Declarative sentences.....: 6
Interrogative sentences.....: 2
Exclamatory sentences.....: 3
Average sentence length (words).....: 6.27
Average word length (chars).....: 4.04
Count of emojis.....: 2
Sorted words by length (preview).....: ['programming', 'interesting', 'identifier', 'cons
onants', 'Sometimes', 'special', 'simple', 'Python', 'system', 'letter', 'vowels', 'Hello',
'there', 'doing', 'today', 'there', 'three', 'words', 'Words', 'equal', 'ready', 'This', 'te
st', 'file', 'love', 'very', 'bugs', 'them', 'Here', 'need', 'some', 'like', 'like', 'mama',
'papa', 'case', 'have', 'How', 'are', 'you', 'are', 'But', 'fix', 'for', 'our', 'cat', 'dog
', 'and', 'box', 'and', 'and', 'Are', 'you', 'Yes', 'Let', 'is', 'in', 'It', 'is', 'we', 'is
', 'We', 'am', 'go', 'a', 'I', 'a', 'I', 's']

[FileManager Info] File 'task2_result.txt' is 1114 bytes.
File 'task2_result.txt' successfully archived to 'task2.zip'.
[ZipManager Info] Archive contains: ['task2_result.txt'].
File 'task2_result.txt' compressed size: 1114 bytes.

```

```

task2.txt
1 Hello there! How are you doing today? This is a simple test file.
2 I love programming in Python :-) It is very interesting.
3 Sometimes there are bugs ;--[[[ But we fix them!
4 Here is a special identifier for our system: {e02fd0e4-00fd-090A-ca30-0d00a0038ba0}.
5 We need some three letter words like cat, dog, and box.
6 Words like mama, papa, and case have equal vowels and consonants.
7 Are you ready? Yes, I am! Let's go.

```

```

task2_result.txt
1 Contains GUID: True
2 Count of 3-letter words: 18
3 Words with equal vowels/consonants: ['9: is', '13: file', '15: love', '17: in', '19: It',
4 Total sentences: 11
5 Declarative sentences: 6
6 Interrogative sentences: 2
7 Exclamatory sentences: 3
8 Average sentence length (words): 6.27
9 Average word length (chars): 4.04
10 Count of emojis: 2
11 Sorted words by length (preview): ['programming', 'interesting', 'identifier', 'consonant
12

```

 task2\_result.txt

 task2.py

 task2.txt

 task2.zip

**Задание 3.** В соответствии с заданием своего варианта доработать программу из ЛР3, используя класс и обеспечить:

а) определение дополнительных параметров среднее арифметическое элементов последовательности, медиана, мода, дисперсия, СКО последовательности;

б) с помощью библиотеки `matplotlib` нарисовать графики разных цветов в одной координатной оси:

– график по полученным данным разложения функции в ряд, представленным в таблице,

– график соответствующей функции, представленной с помощью модуля `math`.

Обеспечить отображение координатных осей, легенды, текста и аннотации.

| $x$ | $n$ | $F(x)$ | $Math F(x)$ | $eps$ |
|-----|-----|--------|-------------|-------|
|     |     |        |             |       |

Здесь  $x$  – значение аргумента,  $F(x)$  – значение функции,  $n$  – количество просуммированных членов ряда,  $Math F(x)$  – значение функции, вычисленное с помощью модуля `math`.

в) сохранить графики в файл

$$\ln(1+x) = \sum_{n=0}^{\infty} (-1)^{n-1} \frac{x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} + \dots, |x| < 1$$

Код задания 3:

```
"""
Brief purpose: Module for Task 3. Taylor series calculation, statistics
with NumPy, and plotting with Matplotlib.

Lab number and title: Lab 4 - Working with files, classes, serializers,
regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 16.03.2026

"""
```

```

import math

import numpy as np

import matplotlib.pyplot as plt

from validation import Validation


class SeriesStatsMixin:

    """Mixin class providing statistical calculations using NumPy
    (Requirement 4)."""

    def calculate_statistics(self, sequence):

        """Calculates mean, median, mode, variance, and standard
        deviation."""

        data = np.array(sequence)

        # 1) Mean

        mean_val = np.mean(data)

        # 2) Median

        median_val = np.median(data)

        # 3) Mode

        values, counts = np.unique(data, return_counts=True)

        mode_val = values[np.argmax(counts)]

        # 4) Variance

        var_val = np.var(data)

        # 5) Standard Deviation

        std_val = np.std(data)

```

```

    return {

        "Mean": mean_val,

        "Median": median_val,

        "Mode": mode_val,

        "Variance": var_val,

        "Standard Deviation": std_val

    }

class BaseSeries(SeriesStatsMixin):

    """Base class for math series calculations. Demonstrates
    inheritance."""

    def __init__(self, x, eps):

        self._x = x

        self._eps = eps

        self._exact_value = self._calculate_exact()

    def _calculate_exact(self):

        """Abstract method for exact value calculation."""

        raise NotImplementedError("Must be overridden in subclasses.")

class LogTaylorSeries(BaseSeries):

```

```

"""Class specific to calculating ln(1+x) using Taylor Series."""

def __init__(self, x, eps):

    super().__init__(x, eps)

    self.n = 0

    self.f_value = 0.0

    self.partial_sums = []

def _calculate_exact(self):

    """Polymorphism: exact value for ln(1+x)."""

    return math.log(1 + self._x)

def _term(self, n):

    """Calculates the n-th term of the series."""

    return ((-1) ** (n - 1)) * (self._x ** n) / n

def calculate_series(self):

    """Calculates the Taylor series and tracks partial sums."""

    max_iter = 500

    current_sum = 0.0

    for i in range(1, max_iter + 1):

        term = self._term(i)

        current_sum += term

        self.partial_sums.append(current_sum)

```

```

        if abs(term) <= self._eps:

            self.n = i

            self.f_value = current_sum

            break

    else:

        self.n = max_iter

        self.f_value = current_sum

        print(f"\nWarning: Max iterations ({max_iter}) reached.")

def get_results(self):

    """Returns the series calculation results."""

    return self._x, self.n, self.f_value, self._exact_value,
self._eps

class PlotService:

    """Service class to handle Matplotlib operations."""

    @staticmethod

    def draw_and_save_plot(target_x, target_n):

        """Draws the function and its Taylor approximation, saves to
file."""

        print("\nGenerating and saving plot...")

        x_vals = np.linspace(-0.95, 0.95, 200)

```

```

y_exact = np.log(1 + x_vals)

# Calculate Taylor approximation for all x_vals up to target_n
y_approx = np.zeros_like(x_vals)

for i in range(1, target_n + 1):

    y_approx += ((-1) ** (i - 1)) * (x_vals ** i) / i

plt.figure(figsize=(10, 6))

# Plotting lines with different colors
plt.plot(x_vals, y_exact, label='Math F(x) = ln(1+x)',
color='blue', linewidth=2)

plt.plot(x_vals, y_approx, label=f'Taylor Series
(n={target_n})', color='red', linestyle='--')

# Adding Coordinate axes and Grid
plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.xlabel('Argument x')
plt.ylabel('Function Value F(x)')
plt.title('Taylor Series Approximation vs Exact Function')
plt.grid(True)

# Adding Legend
plt.legend(loc="upper left")

```

```

# Adding Text

plt.text(-0.9, min(y_exact) * 0.8, "Approximation converges
well\nfor  $|x| < 1$ ",

        fontsize=10, bbox=dict(facecolor='lightyellow',
alpha=0.7))

# Adding Annotation pointing to the user's evaluated point
target_y = math.log(1 + target_x)

plt.annotate(f'Evaluated Point\n(x={target_x:.2f})',

            xy=(target_x, target_y),

            xytext=(target_x - 0.3, target_y + 0.5),

            arrowprops=dict(facecolor='green', shrink=0.05))

# Save to file

filename = 'task3_plot.png'

plt.savefig(filename)

print(f"Plot successfully saved as '{filename}'.")

# Display the plot

plt.show()

```

```
class Service:
```

```
    """Main presentation layer coordinating Task 3 logic."""
```

```
    @staticmethod
```

```

def print_table(x, n, f_val, exact_val, eps):

    """Prints formatted result table."""

    print('\n' + '-' * 75)

    print(f'| {"x":^10} | {"n":^5} | {"F(x)":^15} | {"Math  
F(x)":^15} | {"eps":^10} |')

    print('-' * 75)

    print(f'| {x:^10.4f} | {n:^5} | {f_val:^15.6f} |  
{exact_val:^15.6f} | {eps:^10.6f} |')

    print('-' * 75 + '\n')

    @staticmethod

    def presentation():

        print("\n---[ TASK 3: TAYLOR SERIES & STATISTICS ] ---")

        eps = Validation.inputEpsilon("Enter epsilon (calculation  
accuracy, e.g., 0.001): ")

        x = Validation.inputArgumentX("Enter argument x ( $|x| < 1$ ): ")

        # Calculation

        series_obj = LogTaylorSeries(x, eps)

        series_obj.calculate_series()

        # Display Table

        results = series_obj.get_results()

        Service.print_table(*results)

```

```

# Sequence Statistics (NumPy)

print("--- Statistics of Partial Sums Sequence ---")

stats = series_obj.calculate_statistics(series_obj.partial_sums)

for key, value in stats.items():

    print(f"{key}: {value:.6f}")

# Matplotlib Plot

PlotService.draw_and_save_plot(x, series_obj.n)

```

Демонстрация задания 3:

```

---[ TASK 3: TAYLOR SERIES & STATISTICS ] ---
Enter epsilon (calculation accuracy, e.g., 0.001): 0.00001
Enter argument x ( $|x| < 1$ ): -0.8

```

| x       | n  | F(x)      | Math F(x) | eps      |
|---------|----|-----------|-----------|----------|
| -0.8000 | 36 | -1.609406 | -1.609438 | 0.000010 |

```

--- Statistics of Partial Sums Sequence ---

```

```
Mean: -1.543037
```

```
Median: -1.606590
```

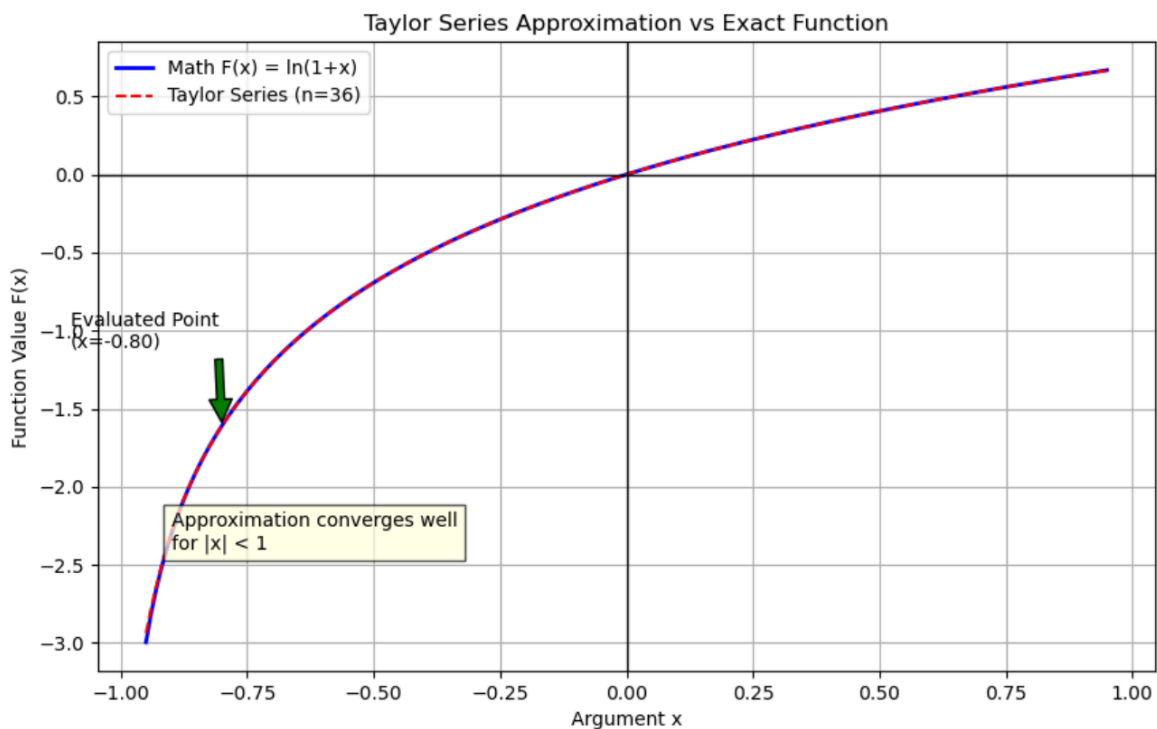
```
Mode: -1.609406
```

```
Variance: 0.025878
```

```
Standard Deviation: 0.160868
```

```
Generating and saving plot...
```

```
Plot successfully saved as 'task3_plot.png'.
```



 task3\_plot.png

 task3.py

**Задание 4.** В соответствии с заданием своего варианта разработать базовые классы и классы наследники. Требования по использованию классов:

Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры (<https://docs.python.org/3/library/abc.html> )

Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры (<https://docs.python.org/3/library/functions.html#property> )

Класс «Прямоугольник» (Круг, Ромб, Квадрат, Треугольник и т.д.) наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» (для другого типа фигуры соответствующие параметры, например, для круга задаем «радиус») и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры <https://docs.python.org/3/library/math.html> .

Для класса «Прямоугольник»(тип фигуры в инд. задании) определить метод, который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Использовать метод `format` (<https://pyformat.info/> ) название фигуры должно задаваться в виде поля данных класса и возвращаться методом класса.

В корневом каталоге проекта создайте файл `main.py` для тестирования классов. Используйте конструкцию, описанную в [https://docs.python.org/3/library/\\_\\_main\\_\\_.html](https://docs.python.org/3/library/__main__.html)

Пример объекта: Прямоугольник синего цвета шириной 5 и высотой 8.

Программа должна содержать следующие базовые функции:

- 1) ввод значений параметров пользователем;
- 2) проверка корректности вводимых данных;
- 3) построение, закрашивание фигуры в выбранный цвет, введенный клавиатуры, и подпись фигуры текстом, введенным с клавиатуры4) вывод фигуры на экран и в файл.

Индивидуальное задание: Построить ромб по стороне  $a$  и тупому углу  $R$  (в градусах).

Код задания 4:

```
"""
Brief purpose: Module for Task 4. OOP implementation of geometric
figures (Rhombus) with drawing capabilities.

Lab number and title: Lab 4 - Working with files, classes, serializers,
regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 17.03.2026

"""

import math

import matplotlib.pyplot as plt

from matplotlib.patches import Polygon

from abc import ABC, abstractmethod

from validation import Validation
```

```

class FigureColor:

    """Class to encapsulate the color of a figure using properties."""

    def __init__(self, color):

        self._color = color

    @property

    def color(self):

        """Getter for the color property."""

        return self._color

    @color.setter

    def color(self, new_color):

        """Setter for the color property."""

        self._color = new_color


class GeometricFigure(ABC):

    """Abstract base class for all geometric figures."""

    @abstractmethod

    def calculate_area(self):

        """Abstract method to calculate the area of the figure."""

        pass

```

```

class Rhombus(GeometricFigure):

    """Class representing a Rhombus. Inherits from GeometricFigure."""

    # Class attribute for the figure name

    _figure_name = "Ромб"

    @classmethod

    def get_figure_name(cls):

        """Class method returning the name of the figure."""

        return cls._figure_name

    def __init__(self, a, r_angle, color):

        """

        Constructor for Rhombus.

        :param a: side length

        :param r_angle: obtuse angle in degrees (90 < R < 180)

        :param color: string representing color

        """

        self.a = a

        self.r_angle = r_angle

        # Creating FigureColor object inside the constructor

        self._color_obj = FigureColor(color)

    @property

    def color(self):

```

```

        """Property to access the color from the internal FigureColor
object."""

        return self._color_obj.color

def calculate_area(self):

    """

    Overrides the abstract method.

    Calculates area of a rhombus: Area = a^2 * sin(angle).

    Uses math module to convert degrees to radians and calculate
sine.

    """

    radians = math.radians(self.r_angle)

    return (self.a ** 2) * math.sin(radians)

def get_info(self):

    """Returns formatted string with basic parameters, color, and
area."""

    area = self.calculate_area()

    # Using the .format() method

    return "{0} {1} цвета со стороной {2} и тупым углом {3}°.
Площадь: {4:.2f}".format(

        self.get_figure_name(),

        self.color,

        self.a,

        self.r_angle,

        area

    )

```

```

class PlotService:

    """Service to handle drawing and saving the figure using
    Matplotlib."""

    @staticmethod

    def draw_rhombus(rhombus_obj, label_text):

        """Calculates vertices of the rhombus and draws it on a plot."""

        print("\nGenerating drawing...")

        # Calculate the acute angle for drawing purposes

        acute_angle_deg = 180 - rhombus_obj.r_angle

        alpha_rad = math.radians(acute_angle_deg)

        a = rhombus_obj.a

        # Calculate vertices (Bottom-Left, Bottom-Right, Top-Right,
        Top-Left)

        # Assuming bottom-left starts at (0,0) and bottom edge lies on X
        axis

        x0, y0 = 0, 0

        x1, y1 = a, 0

        x2, y2 = a + a * math.cos(alpha_rad), a * math.sin(alpha_rad)

        x3, y3 = a * math.cos(alpha_rad), a * math.sin(alpha_rad)

        vertices = [[x0, y0], [x1, y1], [x2, y2], [x3, y3]]

```

```

fig, ax = plt.subplots(figsize=(8, 6))

# Create and add the polygon

polygon = Polygon(vertices, closed=True,
facecolor=rhombus_obj.color, edgecolor='black', linewidth=2)

ax.add_patch(polygon)

# Configure axes limits to ensure the figure is centered and
visible

ax.set_xlim(-0.5 * a, 2.5 * a)

ax.set_ylim(-0.5 * a, 1.5 * a)

ax.set_aspect('equal')

ax.grid(True, linestyle='--', alpha=0.6)

# Add the custom text label in the center of the rhombus

center_x = (x0 + x1 + x2 + x3) / 4

center_y = (y0 + y1 + y2 + y3) / 4

ax.text(center_x, center_y, label_text, ha='center',
va='center',

        fontsize=12, fontweight='bold',
bbox=dict(facecolor='white', alpha=0.7, edgecolor='black'))

# Set title using the formatted info string

plt.title(rhombus_obj.get_info())

# Output to file and screen

```

```

    filename = "task4_rhombus.png"

    plt.savefig(filename)

    print(f"Figure successfully saved to '{filename}'.")

    plt.show()

class Service:

    """Main presentation layer coordinating Task 4 logic."""

    @staticmethod

    def get_valid_side():

        """Validates that the side length is positive."""

        return Validation.inputFigureSide("Enter rhombus side length (a
> 0): ")

    @staticmethod

    def get_valid_angle():

        """Validates that the angle is strictly obtuse (between 90 and
180 degrees)."""

        return Validation.inputFigureAngle("Enter obtuse angle in
degrees (90 < R < 180): ")

    @staticmethod

    def get_valid_color():

        """Validates if the entered string is a recognizable color in
Matplotlib."""

```

```

        return Validation.inputFigureColor("Enter color (e.g., 'red',
'blue', 'green', 'magenta', '#FF5733'): ")

    @staticmethod

    def presentation():

        print("\n---[ TASK 4: GEOMETRIC FIGURES (RHOMBUS) ] ---")

        # User input with strict correctness validation

        side_a = Service.get_valid_side()

        angle_r = Service.get_valid_angle()

        color = Service.get_valid_color()

        label_text = Validation.inputStr("Enter a text label for the
figure: ")

        # Construct object

        my_rhombus = Rhombus(side_a, angle_r, color)

        # Print info

        print("\nFigure Info:")

        print(my_rhombus.get_info())

        # Draw, label, color, output to screen and file

        PlotService.draw_rhombus(my_rhombus, label_text)

```

#### Демонстрация задания 4:

```
---[ TASK 4: GEOMETRIC FIGURES (RHOMBUS) ] ---
```

```
Enter rhombus side length (a > 0): 13
```

```
Enter obtuse angle in degrees (90 < R < 180): 155
```

```
Enter color (e.g., 'red', 'blue', 'green', 'magenta', '#FF5733'): cyan
```

```
Enter a text label for the figure: MyRombus
```

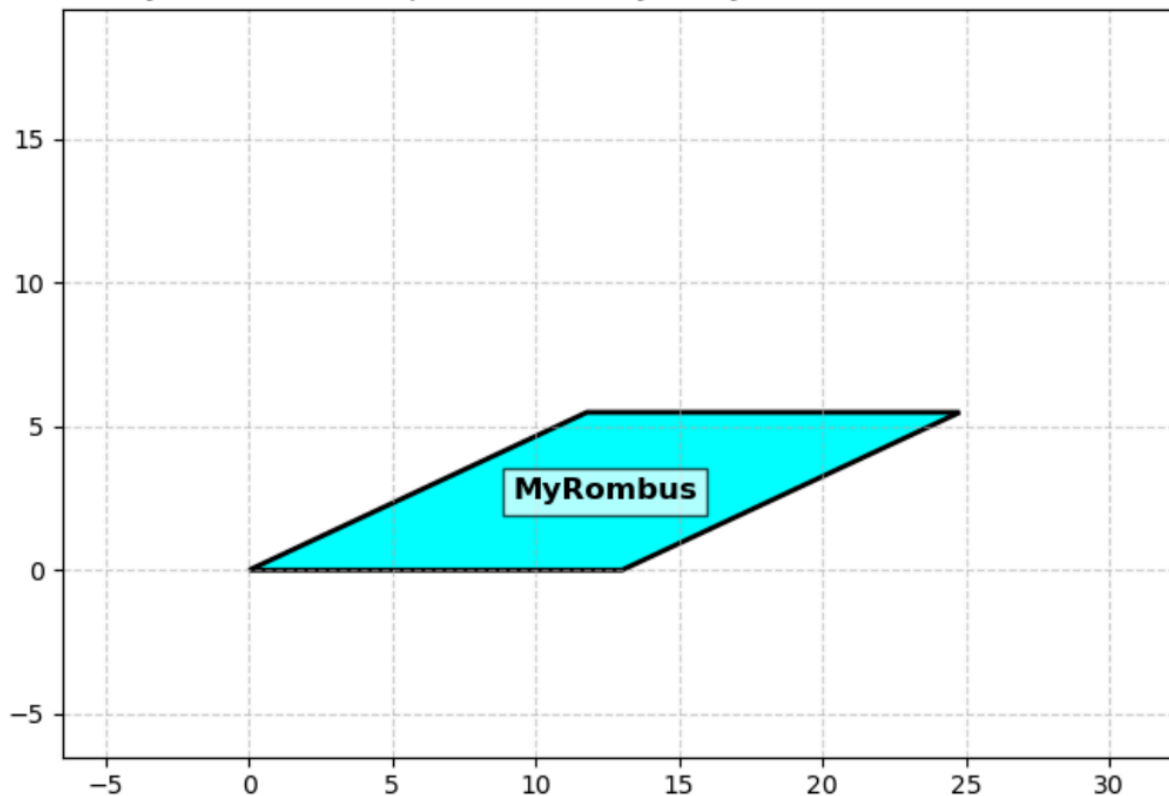
Figure Info:

Ромб с cyan цвета со стороной 13.0 и тупым углом 155.0°. Площадь: 71.42

Generating drawing...

Figure successfully saved to 'task4\_rhombus.png'.

Ромб с cyan цвета со стороной 13.0 и тупым углом 155.0°. Площадь: 71.42



 task4\_rhombus.png

 task4.py

**Задание 5.** В соответствии с заданием своего варианта исследовать возможности библиотека NumPy при работе с массивами и математическими и статическими операциями. Сформировать целочисленную матрицу  $A[n,m]$  с помощью генератора случайных чисел (random).

а) Библиотека NumPy.

1. Создание массива. Функции `array()` и `values()`. 2. Функции создания массива заданного вида.

3. Индексирование массивов NumPy. Индекс и срез.

4. Операции с массивами. Универсальные (поэлементные) функции.

б) Математические и статистические операции.

1. Функция `mean()`

2. Функция `median()`

3. Функция `corrcoef()`

4. Дисперсия `var()`.

5. Стандартное отклонение `std()`

Индивидуальное задание: Отсортировать матрицу по убыванию элементов последнего столбца. Вычислить среднее значение элементов последнего столбца. Ответ округлите до сотых. Вычисление среднего значения выполнить двумя способами: через стандартную функцию и через программирование формулы

Код задания 5:

```
"""
Brief purpose: Module for Task 5. Exploring NumPy capabilities and
sorting a random matrix.

Lab number and title: Lab 4 - Working with files, classes, serializers,
regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 17.03.2026
"""

import numpy as np
```

```

from validation import Validation

class NumpyDemonstrationMixin:
    """
    Mixin class to demonstrate basic NumPy capabilities (General Task).
    Demonstrates array creation, indexing, slicing, and statistical
    operations.
    """

    def demonstrate_numpy_features(self, matrix):
        """Demonstrates general NumPy features requested in the
        assignment."""

        print("\n" + "="*50)

        print("---[ NUMPY CAPABILITIES DEMONSTRATION ] ---")

        # Creation: array() and values()
        sample_dict = {'x': 10, 'y': 20, 'z': 30}
        arr_from_values = np.array(list(sample_dict.values()))
        print(f"1. np.array() from dict values(): {arr_from_values}")

        # Specific arrays
        zeros_arr = np.zeros((2, 2), dtype=int)
        ones_arr = np.ones((2, 2), dtype=int)

        print(f"2. Specific arrays\n(zeros):\n{zeros_arr}\n(ones):\n{ones_arr}")

        # Indexing and slicing
        if matrix.shape[0] > 1 and matrix.shape[1] > 1:
            print(f"3. Indexing (element [0, 0]): {matrix[0, 0]}")

```

```

        print(f"    Slicing (first row): {matrix[0, :]}" )

        print(f"    Slicing (last column): {matrix[:, -1]}" )

        # Element-wise universal functions

        print("4. Element-wise operations (Matrix + 100):\n", matrix +
100)

        print("\n---[ MATH & STATS OPERATIONS DEMONSTRATION ] ---")

        # Demonstrating stats on the generated matrix

        print(f"1. Mean (np.mean): {np.mean(matrix):.2f}" )

        print(f"2. Median (np.median): {np.median(matrix):.2f}" )

        # Calculate correlation between the rows of the matrix

        correlation = np.corrcoef(matrix)

        print("3. Correlation coefficient (np.corrcoef) preview (first
2x2):\n", correlation[:2, :2])

        print(f"4. Variance (np.var): {np.var(matrix):.2f}" )

        print(f"5. Standard Deviation (np.std): {np.std(matrix):.2f}" )

        print("="*50 + "\n")

class BaseMatrix(NumpyDemonstrationMixin):

    """Base class handling the creation and storage of the random
matrix."""

    def __init__(self, n, m):

        """Initializes dimensions and generates random matrix A[n,
m]."""

        self._n = n

```

```

        self._m = m

        # Generating integer matrix using random generator

        self._matrix = np.random.randint(10, 100, size=(n, m))

    @property
    def matrix(self):

        """Property to access the encapsulated matrix."""

        return self._matrix

    def __str__(self):

        """String representation of the matrix."""

        return str(self._matrix)

class VariantMatrix(BaseMatrix):

    """Child class to execute the individual variant task."""

    def __init__(self, n, m):

        super().__init__(n, m)

    def get_sorted_by_last_column(self):

        """

        Sorts the matrix in descending order based on the last column.

        """

        # Get the last column

        last_col = self._matrix[:, -1]

        # Get sorted indices in ascending order, then reverse them
        [::-1] for descending

        sorted_indices = np.argsort(last_col)[::-1]

```

```

        # Return new matrix sliced by sorted indices

        return self._matrix[sorted_indices]

    def get_last_col_mean_standard(self):

        """Calculates the mean of the last column using standard NumPy
        function."""

        last_col = self._matrix[:, -1]

        mean_val = np.mean(last_col)

        return round(mean_val, 2)

    def get_last_col_mean_manual(self):

        """Calculates the mean of the last column using a manually
        programmed formula."""

        last_col = self._matrix[:, -1]

        total_sum = 0

        for value in last_col:

            total_sum += value

        mean_val = total_sum / len(last_col)

        return round(mean_val, 2)

class Service:

    """Main presentation layer coordinating Task 5 logic."""

    @staticmethod

    def get_valid_dimension(prompt):

        """Validates that matrix dimensions are at least 2."""

        return Validation.inputMatrixDimension(prompt)

```

```

@staticmethod

def presentation():

    print("\n---[ TASK 5: NUMPY MATRICES & OPERATIONS ] ---")

    # User input for Matrix size

    n = Service.get_valid_dimension("Enter number of rows (n >= 2): ")

    m = Service.get_valid_dimension("Enter number of columns (m >= 2): ")

    # Create the Variant Matrix Object

    task_matrix = VariantMatrix(n, m)

    # Demonstrate General NumPy Features

    task_matrix.demonstrate_numpy_features(task_matrix.matrix)

    # --- Individual Task Demonstration ---

    print("--- [ INDIVIDUAL TASK RESULTS ] ---")

    print("1. Original Matrix A[n, m]:")

    print(task_matrix)

    sorted_mat = task_matrix.get_sorted_by_last_column()

    print("\n2. Matrix sorted in DESCENDING order of the LAST column:")

    print(sorted_mat)

    mean_std = task_matrix.get_last_col_mean_standard()

    mean_man = task_matrix.get_last_col_mean_manual()

```

```
print("\n3. Average of the last column:")

print(f"    -> Using standard function (np.mean): {mean_std}")
print(f"    -> Using programmed formula (sum/len): {mean_man}")

# Double check if both methods yield the same result

if mean_std == mean_man:
    print("\n[SUCCESS] Both average calculation methods match!")
```

Демонстрация задания 5:

```

---[ TASK 5: NUMPY MATRICES & OPERATIONS ] ---
Enter number of rows (n >= 2): 5
Enter number of columns (m >= 2): 5

=====
---[ NUMPY CAPABILITIES DEMONSTRATION ] ---
1. np.array() from dict values(): [10 20 30]
2. Specific arrays (zeros):
[[0 0]
 [0 0]]
(ones):
[[1 1]
 [1 1]]
3. Indexing (element [0, 0]): 68
   Slicing (first row): [68 15 25 26 33]
   Slicing (last column): [33 37 78 20 78]
4. Element-wise operations (Matrix + 100):
[[168 115 125 126 133]
 [169 166 136 198 137]
 [137 182 192 182 178]
 [148 136 110 171 120]
 [146 143 115 136 178]]

---[ MATH & STATS OPERATIONS DEMONSTRATION ] ---
1. Mean (np.mean): 49.88
2. Median (np.median): 43.00
3. Correlation coefficient (np.corrcoef) preview (first 2x2):
[[1.          0.06214259]
 [0.06214259 1.          ]]
4. Variance (np.var): 660.03
5. Standard Deviation (np.std): 25.69
=====

```

```

--- [ INDIVIDUAL TASK RESULTS ] ---
1. Original Matrix A[n, m]:
[[68 15 25 26 33]
 [69 66 36 98 37]
 [37 82 92 82 78]
 [48 36 10 71 20]
 [46 43 15 36 78]]

2. Matrix sorted in DESCENDING order of the LAST column:
[[46 43 15 36 78]
 [37 82 92 82 78]
 [69 66 36 98 37]
 [68 15 25 26 33]
 [48 36 10 71 20]]

3. Average of the last column:
  -> Using standard function (np.mean): 49.2
  -> Using programmed formula (sum/len): 49.2

[SUCCESS] Both average calculation methods match!

```

**Задание 6.** Используя указанный в задании набор данных (datasets) на ресурсе <https://www.kaggle.com/datasets> исследовать основные возможности библиотеки Pandas. Если какая-то из ссылок не работает, то искать на ресурсе с помощью поиска или в google (пример запроса: spotify dataset kaggle). См. Анализ Данных Python методичка.pdf.

#### ЗАДАНИЕ А. Библиотека Pandas. Структуры Series и DataFrame

1. Библиотека Pandas. Импорт библиотеки.
2. Структура Series.
3. Создание Series .
4. Функция display.
5. Доступ к элементам Series с использованием .loc или .iloc.
6. Объект DataFrame. Создание.

| № | Датасет | Ссылка<br>на<br>Kaggle | Конкретное задание | Ключевые<br>операции |
|---|---------|------------------------|--------------------|----------------------|
|---|---------|------------------------|--------------------|----------------------|

|    |                   |                                       |                                                                                                                                     |                                                         |
|----|-------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| 23 | Кибербезопасность | <a href="#">Cybersecurity Attacks</a> | Создайте пустой DataFrame для атак. Добавьте 3 столбца: 'attack_type', 'severity', 'duration'. Добавьте 2 строки данных из dataset. | Создание пустого DataFrame, добавление столбцов и строк |
|----|-------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|

#### ЗАДАНИЕ Б. Основные операции и статистический анализ

Получение информации о датафрейме (по каждому параметру). Индексация и извлечение данных: статистические методы (во сколько раз среднее значение выбранного показателя с максимальным значением другого параметра (например, самый молодой, самый быстрый и т.д.) выше среднего выбранного показателя с минимальным значением).

| №  | Датасет           | Ссылка на Kaggle                      | Статистическое задание (формат: "Определите, во сколько раз...")                                                                                                                           |
|----|-------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 23 | Кибербезопасность | <a href="#">Cybersecurity Attacks</a> | Определите, во сколько раз средняя продолжительность атаки (Duration) атак максимальной серьезности (Severity = max) больше, чем у атак минимальной серьезности. Ответ округлите до сотых. |

Код задания 6:

```
"""
Brief purpose: Module for Task 6. Data analysis using Pandas (Series,
DataFrame, Statistics).

Lab number and title: Lab 4 - Working with files, classes, serializers,
regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 18.03.2026
"""

import os

import pandas as pd

from abc import ABC, abstractmethod
```

```

try:
    from IPython.display import display
except ImportError:
    display = print

class PandasDemoMixin:
    """Mixin class to demonstrate basic Pandas features (Part A
    General)."""

    def demonstrate_series_features(self):
        """Demonstrates Series creation, display, .loc, and .iloc
        operations."""

        print("\n--- [ PART A: PANDAS CAPABILITIES DEMONSTRATION ] ---")

        # Creating a Series
        data_list = [404, 502, 200]

        index_labels = ['Not Found', 'Bad Gateway', 'OK']

        my_series = pd.Series(data_list, index=index_labels,
                               name="HTTP_Codes")

        print("\n[Created Series object]:")

        # The display function
        display(my_series)

        # Access via .loc and .iloc
        print("\n[Accessing elements in Series]:")

        print(f"Using .loc['Bad Gateway'] (by label):
{my_series.loc['Bad Gateway']}")

        print(f"Using .iloc[0] (by index 0): {my_series.iloc[0]}")

```

```

class BaseDataAnalyzer(ABC):

    """Abstract base class for Data Analysis. Demonstrates OOP and
    ABC."""

    def __init__(self, filepath):

        self._filepath = filepath

        self._df = None

    @property

    def dataframe(self):

        """Property getter for the loaded dataframe."""

        return self._df

    @abstractmethod

    def load_data(self):

        """Abstract method to load data from the source."""

        pass

class CyberSecurityAnalyzer(BaseDataAnalyzer, PandasDemoMixin):

    """Class handling the Cyber Security Dataset logic. Demonstrates
    Inheritance and Mixins."""

    def __init__(self, filepath):

        super().__init__(filepath)

    def load_data(self):

```

```

        """Polymorphism: Overrides load_data to read CSV
specifically."""

        if os.path.exists(self._filepath):

            self._df = pd.read_csv(self._filepath)

            print(f"\n[Success] Dataset '{self._filepath}' loaded
successfully. Rows: {len(self._df)}")

        else:

            print(f"\n[Error] File '{self._filepath}' not found! Make
sure it is located next to the script.")

            self._df = pd.DataFrame() # Fallback to empty dataframe

def part_a_individual(self):

    """

    Executes Part A Individual Task:

    Create empty DataFrame, add 3 columns, add 2 rows from the
dataset.

    """

    print("\n--- [ PART A (Individual task): EMPTY DATAFRAME ] ---")

    # Creating an empty DataFrame and Adding 3 columns

    empty_df = pd.DataFrame(columns=['attack_type', 'severity',
'duration'])

    print("\n[Empty DataFrame created with specified columns]:")

    display(empty_df)

    # Adding 2 data rows from the loaded dataset

    if self._df is not None and not self._df.empty and len(self._df)
>= 2:

        row1 = self._df.iloc[0]

        row2 = self._df.iloc[1]

```

```

        # Extract the required columns: Attack Type, Severity Level,
        Packet Length

        empty_df.loc[0] =[row1['Attack Type'], row1['Severity
Level'], row1['Packet Length']]

        empty_df.loc[1] = [row2['Attack Type'], row2['Severity
Level'], row2['Packet Length']]

        print("\n[DataFrame after adding 2 rows from the Kaggle
dataset]:")

        display(empty_df)

    else:

        print("\n[Warning] Main dataset is not loaded. Cannot add
rows.")

def part_b_general(self):

    """Executes Part B General Task: DataFrame information and basic
stats."""

    print("\n--- [ PART B (General task): DATAFRAME INFORMATION ]
---")

    if self._df is None or self._df.empty:

        print("No data for analysis.")

        return

    print("\n[DataFrame information (.info())]:")

    self._df.info()

    print("\n[Basic statistics for numerical columns
(.describe())]:")

    display(self._df.describe())

def part_b_individual(self):

```

```

"""
Executes Part B Individual Task:

Ratio of average duration (Packet Length) of MAX severity
("High") vs MIN severity ("Low").

"""

print("\n--- [ PART B (Individual task): STATISTICAL ANALYSIS ]
---")

if self._df is None or self._df.empty:

    print("No data for analysis.")

    return

try:

    # Extract data using boolean indexing and calculate the mean

    mean_duration_max = self._df[self._df['Severity Level'] ==
'High']['Packet Length'].mean()

    mean_duration_min = self._df[self._df['Severity Level'] ==
'Low']['Packet Length'].mean()

    print(f"Max severity ('High') -> Average 'Packet Length':
{mean_duration_max:.2f}")

    print(f"Min severity ('Low') -> Average 'Packet Length':
{mean_duration_min:.2f}")

    if pd.isna(mean_duration_max) or pd.isna(mean_duration_min):

        print("Data is missing for one of the severity levels in
the dataset.")

        return

    if mean_duration_min == 0:

        print("Error: Average duration of minimal attacks is 0
(Division by zero).")

    else:

```

```

        ratio = mean_duration_max / mean_duration_min

        # Round to 2 decimal places

        print(f"\n[RESULT]: The average metric of max severity
attacks is {round(ratio, 2)} times greater than the min severity
attacks.")

    except KeyError as e:

        print(f"Error: Required column {e} not found in the
dataset.")

    except Exception as e:

        print(f"An unexpected error occurred: {e}")

class Service:

    """Main presentation layer coordinating Task 6 logic."""

    @staticmethod
    def presentation():

        print("\n" + "="*50)

        print(f"{'TASK 6: PANDAS & KAGGLE DATASET':^50}")

        print("="*50)

        # Name of the downloaded dataset

        filepath = "cybersecurity_attacks.csv"

        analyzer = CyberSecurityAnalyzer(filepath)

        analyzer.load_data()

        # Execute Part A

        analyzer.demonstrate_series_features()

```

```

analyzer.part_a_individual()

# Execute Part B

analyzer.part_b_general()

analyzer.part_b_individual()

print("\n" + "="*50)

```

### Демонстрация задания 6:

```

=====
TASK 6: PANDAS & KAGGLE DATASET
=====

[Success] Dataset 'cybersecurity_attacks.csv' loaded successfully. Rows: 40000

--- [ PART A: PANDAS CAPABILITIES DEMONSTRATION ] ---

[Created Series object]:
Not Found      404
Bad Gateway    502
OK             200
Name: HTTP_Codes, dtype: int64

[Accessing elements in Series]:
Using .loc['Bad Gateway'] (by label): 502
Using .iloc[0] (by index 0): 404

--- [ PART A (Individual task): EMPTY DATAFRAME ] ---

[Empty DataFrame created with specified columns]:
Empty DataFrame
Columns: [attack_type, severity, duration]
Index: []

[DataFrame after adding 2 rows from the Kaggle dataset]:
  attack_type severity  duration
0    Malware      Low      503
1    Malware      Low     1174

```

--- [ PART B (General task): DATAFRAME INFORMATION ] ---

```
[DataFrame information (.info())]:  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 40000 entries, 0 to 39999  
Data columns (total 25 columns):
```

| #  | Column                 | Non-Null Count | Dtype   |
|----|------------------------|----------------|---------|
| 0  | Timestamp              | 40000 non-null | object  |
| 1  | Source IP Address      | 40000 non-null | object  |
| 2  | Destination IP Address | 40000 non-null | object  |
| 3  | Source Port            | 40000 non-null | int64   |
| 4  | Destination Port       | 40000 non-null | int64   |
| 5  | Protocol               | 40000 non-null | object  |
| 6  | Packet Length          | 40000 non-null | int64   |
| 7  | Packet Type            | 40000 non-null | object  |
| 8  | Traffic Type           | 40000 non-null | object  |
| 9  | Payload Data           | 40000 non-null | object  |
| 10 | Malware Indicators     | 20000 non-null | object  |
| 11 | Anomaly Scores         | 40000 non-null | float64 |
| 12 | Alerts/Warnings        | 19933 non-null | object  |
| 13 | Attack Type            | 40000 non-null | object  |
| 14 | Attack Signature       | 40000 non-null | object  |
| 15 | Action Taken           | 40000 non-null | object  |
| 16 | Severity Level         | 40000 non-null | object  |
| 17 | User Information       | 40000 non-null | object  |
| 18 | Device Information     | 40000 non-null | object  |
| 19 | Network Segment        | 40000 non-null | object  |
| 20 | Geo-location Data      | 40000 non-null | object  |
| 21 | Proxy Information      | 20149 non-null | object  |
| 22 | Firewall Logs          | 20039 non-null | object  |
| 23 | IDS/IPS Alerts         | 19950 non-null | object  |

```
24 Log Source          40000 non-null object
dtypes: float64(1), int64(3), object(21)
memory usage: 7.6+ MB
```

```
[Basic statistics for numerical columns (.describe())]:
count    Source Port    Destination Port    Packet Length    Anomaly Scores
mean      32970.356450    33150.868650       781.452725      50.113473
std       18560.425604    18574.668842      416.044192      28.853598
min       1027.000000     1024.000000       64.000000       0.000000
25%      16850.750000     17094.750000      420.000000      25.150000
50%      32856.000000     33004.500000      782.000000      50.345000
75%      48928.250000     49287.000000     1143.000000      75.030000
max       65530.000000     65535.000000     1500.000000     100.000000
```

```
--- [ PART B (Individual task): STATISTICAL ANALYSIS ] ---
Max severity ('High') -> Average 'Packet Length': 780.83
Min severity ('Low') -> Average 'Packet Length': 779.92
```

[RESULT]: The average metric of max severity attacks is 1.0 times greater than the min severity attacks.

cybersecurity\_attacks.csv > data

|    | Timestamp           | Source IP Address | Destination IP Address | Source Port | Destination Port | Protocol | Alert                     | Severity | Pattern |
|----|---------------------|-------------------|------------------------|-------------|------------------|----------|---------------------------|----------|---------|
| 1  | 2023-05-30 06:33:58 | 103.216.15.12     | 84.9.164.252           | 31225       | 17616            | ICMP     | 503,Data,HTTP,"Qui natu   |          |         |
| 2  | 2023-05-30 06:33:58 | 103.216.15.12     | 84.9.164.252           | 31225       | 17616            | ICMP     | 503,Data,HTTP,"Qui natu   |          |         |
| 3  | 2020-08-26 07:08:30 | 78.199.217.198    | 66.191.137.154         | 17245       | 48166            | ICMP     | 1174,Data,HTTP,"Aper      |          |         |
| 4  | 2020-08-26 07:08:30 | 78.199.217.198    | 66.191.137.154         | 17245       | 48166            | ICMP     | 1174,Data,HTTP,"Aper      |          |         |
| 5  | 2022-11-13 08:23:25 | 63.79.210.48      | 198.219.82.17          | 16811       | 53600            | UDP      | 306,Control,HTTP,Perfere  |          |         |
| 6  | 2023-07-02 10:38:46 | 163.42.196.10     | 101.228.192.255        | 20018       | 32534            | UDP      | 385,Data,HTTP,Totam m     |          |         |
| 7  | 2023-07-16 13:11:07 | 71.166.185.76     | 189.243.174.238        | 6131        | 26646            | TCP      | 1462,Data,DNS,"Odit ne    |          |         |
| 8  | 2023-07-16 13:11:07 | 71.166.185.76     | 189.243.174.238        | 6131        | 26646            | TCP      | 1462,Data,DNS,"Odit ne    |          |         |
| 9  | 2022-10-28 13:14:27 | 198.102.5.160     | 147.190.155.133        | 17430       | 52805            | UDP      | 1423,Data,HTTP,Repell     |          |         |
| 10 | 2022-05-16 17:55:43 | 97.253.103.59     | 77.16.101.53           | 26562       | 17416            | TCP      | 379,Data,DNS,"Qui numqua  |          |         |
| 11 | 2022-05-16 17:55:43 | 97.253.103.59     | 77.16.101.53           | 26562       | 17416            | TCP      | 379,Data,DNS,"Qui numqua  |          |         |
| 12 | 2023-02-12 07:13:17 | 11.48.99.245      | 178.157.14.116         | 34489       | 20396            | ICMP     | 1022,Data,DNS,"Amet li    |          |         |
| 13 | 2023-02-12 07:13:17 | 11.48.99.245      | 178.157.14.116         | 34489       | 20396            | ICMP     | 1022,Data,DNS,"Amet li    |          |         |
| 14 | 2023-06-27 11:02:56 | 49.32.208.167     | 72.202.237.9           | 56296       | 20857            | TCP      | 1281,Control,FTP,Veritat  |          |         |
| 15 | 2021-08-15 22:29:04 | 114.109.149.113   | 160.88.194.172         | 37918       | 50039            | UDP      | 224,Data,HTTP,Conseq      |          |         |
| 16 | 2022-07-20 13:28:50 | 177.21.83.200     | 196.218.124.169        | 35538       | 35006            | ICMP     | 661,Data,HTTP,"Sequi      |          |         |
| 17 | 2022-07-20 13:28:50 | 177.21.83.200     | 196.218.124.169        | 35538       | 35006            | ICMP     | 661,Data,HTTP,"Sequi      |          |         |
| 18 | 2022-06-26 15:15:50 | 92.4.25.171       | 112.43.185.24          | 10903       | 36817            | TCP      | 281,Control,HTTP,Nihil pr |          |         |
| 19 | 2020-09-30 21:35:31 | 57.91.207.84      | 98.96.110.38           | 53471       | 38048            | ICMP     | 64,Control,DNS,Earum sit  |          |         |
| 20 | 2021-01-13 02:30:18 | 80.28.21.123      | 111.204.103.106        | 43729       | 10845            | ICMP     | 1341,Data,HTTP,"Ab vo     |          |         |
| 21 | 2021-01-13 02:30:18 | 80.28.21.123      | 111.204.103.106        | 43729       | 10845            | ICMP     | 1341,Data,HTTP,"Ab vo     |          |         |
| 22 | 2023-02-01 13:17:17 | 54.163.130.178    | 62.112.149.214         | 47795       | 35243            | UDP      | 913,Data,DNS,"Sit dol     |          |         |
| 23 | 2023-02-01 13:17:17 | 54.163.130.178    | 62.112.149.214         | 47795       | 35243            | UDP      | 913,Data,DNS,"Sit dol     |          |         |
| 24 | 2023-09-15 20:35:21 | 130.163.192.252   | 12.192.2.112           | 24912       | 21176            | TCP      | 425,Data,DNS,Voluptate    |          |         |
| 25 | 2023-09-15 20:35:21 | 130.163.192.252   | 12.192.2.112           | 24912       | 21176            | TCP      | 425,Data,DNS,Voluptate    |          |         |
| 26 | 2023-04-18 16:46:46 | 4.255.187.165     | 136.159.186.239        | 39934       | 5259             | TCP      | 838,Control,HTTP,Sed f    |          |         |
| 27 | 2023-04-18 16:46:46 | 4.255.187.165     | 136.159.186.239        | 39934       | 5259             | TCP      | 838,Control,HTTP,Sed f    |          |         |
| 28 | 2023-02-21 07:02:55 | 212.164.196.41    | 32.26.31.49            | 16513       | 50583            | TCP      | 969,Data,HTTP,"Dolore ni  |          |         |
| 29 | 2023-02-21 07:02:55 | 212.164.196.41    | 32.26.31.49            | 16513       | 50583            | TCP      | 969,Data,HTTP,"Dolore ni  |          |         |

Также в отдельных модулях были реализованы проверка ввода с декоратором и обработкой конкретных исключений, возможность запуска задания заново без выхода из программы и управление всеми заданиями также в отдельном модуле:

#### 1) Проверка ввода:

```
''' '''

Brief purpose: Module for user input validation and exception
handling.
```

Lab number and title: Lab 4 - Working with files, classes, serializers, regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 15.03.2026

"""

```
import matplotlib.colors as mcolors
```

```
class Validation:
```

```
    """Class containing static methods for validating user
    input."""
```

```
    @staticmethod
```

```
    def userInputNum(func):
```

```
        """
```

```
        Decorator to handle ValueError exceptions during user
        input.
```

```
        Keeps prompting the user until valid input is provided.
```

```
        """
```

```
        def wrapper(prompt):
```

```
            while True:
```

```
                try:
```

```
                    return func(prompt)
```

```
                except ValueError:
```

```
                    print("Error: Wrong input, please try
                    again!\n")
```

```
            return wrapper
```

```
    @staticmethod
```

```

@userInputNum

def inputIntNum(prompt):

    """Prompts the user for an integer number and returns
it."""

    return int(input(prompt))


@staticmethod

@userInputNum

def inputFloatNum(prompt):

    """Prompts the user for a floating-point number and
returns it."""

    return float(input(prompt))


@staticmethod

@userInputNum

def inputStr(prompt):

    """Prompts the user for a string, ensuring it is not
empty."""

    user_str = input(prompt).strip()

    if not user_str:

        print("Error: String cannot be empty.")

        raise ValueError()

    return user_str


@staticmethod

@userInputNum

def inputEpsilon(prompt):

    """

    Requests an epsilon (accuracy) value greater than 0.

    """

```

```

    eps = Validation.inputFloatNum(prompt)

    if eps <= 0:
        print("Epsilon must be greater than 0.")
        raise ValueError()

    return eps


@staticmethod
@userInputNum
def inputArgumentX(prompt):
    """
    Requests an argument X where  $|X| < 1$ .
    """
    x = Validation.inputFloatNum(prompt)

    if abs(x) >= 1:
        print("Absolute value of X must be less than 1.")
        raise ValueError()

    return x


@staticmethod
@userInputNum
def inputFigureSide(prompt):
    """
    Requests a side length of figure where length > 0
    """
    side = Validation.inputFloatNum(prompt)

    if side <= 0:
        print("Error: Side length must be strictly greater
than 0.\n")

```

```

        raise ValueError()

    return side

    @staticmethod
    @userInputNum
    def inputFigureAngle(prompt):
        """
        Requests an angle of figure where 90 < angle < 180
        """
        angle = Validation.inputFloatNum(prompt)
        if 90 < angle < 180:
            return angle

        print("Error: The angle must be obtuse (between 90 and 180
degrees).\n")
        raise ValueError()

    @staticmethod
    @userInputNum
    def inputFigureColor(prompt):
        """
        Requests a color of figure where color is exist
        """
        color = Validation.inputStr(prompt).lower()
        if mcolors.is_color_like(color):
            return color

        print(f"Error: '{color}' is not recognized as a valid
color. Please try again.\n")
        raise ValueError()

    @staticmethod

```

```

@userInputNum

def inputMatrixDimension(prompt):
    """
    Requests a dimension of matrix where dimension >= 2
    """
    val = Validation.inputIntNum(prompt)

    if val >= 2:
        return val

    print("Error: Matrix dimension must be at least 2 for a
proper demonstration.\n")

    raise ValueError()

```

## 2) Запуск заданий заново без выхода:

```

"""
Brief purpose: Module for handling the console menu logic and
repeating tasks.

Lab number and title: Lab 4 - Working with files, classes,
serializers, regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 15.03.2026

"""

from validation import Validation

class Menu:

    """Class responsible for menu loops and task repetition."""

```

```

    @staticmethod

    def main_loop(task_func):

        """

        Executes the provided task function and asks the user if
they want to

        repeat it or exit to the main menu.

        """

        while True:

            task_func()

            print("\nOptions:")
            print("1) Repeat the task")
            print("2) Exit to main menu")

            while True:

                choice = Validation.inputIntNum("Enter your choice
(1 or 2): ")

                if choice == 1:

                    print("\n" + "="*50 + "\n")

                    break

                elif choice == 2:

                    return

                else:

                    print("Incorrect choice, please try again.")

```

### 3) Управлениями всеми заданиями из другого модуля, тестирование:

```

"""

```

Brief purpose: Main entry point of the application, connecting all task modules.

Lab number and title: Lab 4 - Working with files, classes, serializers, regular expressions, and standard libraries

Version: 1.0

Developer: Romanov Nikolay Viktorovich

Date: 15.03.2026

"""

```
from validation import Validation
```

```
from menu import Menu
```

```
from task1 import Service as task1
```

```
from task2 import Service as task2
```

```
from task3 import Service as task3
```

```
from task4 import Service as task4
```

```
from task5 import Service as task5
```

```
from task6 import Service as task6
```

```
class Main:
```

```
    """Main class to run the application."""
```

```
    @staticmethod
```

```
    def main():
```

```
        """Displays the main menu and handles routing to specific tasks."""
```

```
        while True:
```

```
            print("\n" + "="*40)
```

```
            print(f"{'MAIN MENU':^40}")
```

```
            print("="*40)
```

```
            print("1) Run Task 1")
```

```

print("2) Run Task 2")

print("3) Run Task 3")

print("4) Run Task 4")

print("5) Run Task 5")

print("6) Run Task 6")

print("0) Exit Program")

print("="*40)

choice = Validation.inputIntNum("Enter task number
(0-6): ")

match choice:

    case 1:

        Menu.main_loop(task1.presentation)

    case 2:

        Menu.main_loop(task2.presentation)

    case 3:

        Menu.main_loop(task3.presentation)

    case 4:

        Menu.main_loop(task4.presentation)

    case 5:

        Menu.main_loop(task5.presentation)

    case 6:

        Menu.main_loop(task6.presentation)

    case 0:

        break

    case _:

        print("Invalid choice. Please select an
existing task.")

```

```
if __name__ == "__main__":  
    Main.main()
```